

A Little C Primer/C File-IO Through System Calls

File-I/O through system calls is simpler and operates at a lower level than making calls to the C file-I/O library. There are seven fundamental file-I/O system calls:

<code>creat()</code>	Create a file for reading or writing.
<code>open()</code>	Open a file for reading or writing.
<code>close()</code>	Close a file after reading or writing.
<code>unlink()</code>	Delete a file.
<code>write()</code>	Write bytes to file.
<code>read()</code>	Read bytes from file.

These calls were devised for the UNIX operating system and are not part of the ANSI C spec.

Use of these system calls requires a header file named "fcntl.h":

```
#include <fcntl.h>
```

The "creat()" system call, of course, creates a file. It has the syntax:

```
<file descriptor variable> = creat( <filename>, <protection bits> );
```

This system call returns an integer, called a "file descriptor", which is a number that identifies the file generated by "creat()". This number is used by other system calls in the program to access the file. Should the "creat()" call encounter an error, it will return a file descriptor value of -1.

The "filename" parameter gives the desired filename for the new file. The "permission bits" give the "access rights" to the file. A file has three

"permissions" associated with it:

- Write permission:

Allows data to be written to the file.

- Read permission:

Allows data to be read from the file.

- Execute permission:

Designates that the file is a program that can be run.

These permissions can be set for three different levels:

- User level:

Permissions apply to individual user.

- Group level:

Permissions apply to members of user's defined "group".

- System level:

Permissions apply to everyone on the system.

For the "creat()" system call, the permissions are expressed in octal, with an octal digit giving the three permission bits for each level of permissions. In octal, the permission settings:

0644

—grant read and write permissions for the user, but only read permissions

for group and system. The following octal number gives all permissions to everyone:

0777

An attempt to "creat()" an existing file (for which the program has write permission) will *not* return an error. It will instead wipe the contents of the file and return a file descriptor for it.

For example, to create a file named "data" with read and write permission for everyone on the system would require the following statements:

```
#define RD_WR 0666
...
int fd;                               /Define file descriptor. */
fd = creat( "data", RD_WR );
```

The "open()" system call opens an existing file for reading or writing. It has the syntax:

```
<file descriptor variable> = open( <filename>, <access mode> );
```

The "open()" call is similar to the "creat()" call in that it returns a file descriptor for the given file, and returns a file descriptor of -1 if it encounters an error. However, the second parameter is an "access mode", not a permission code. There are three modes (defined in the "fcntl.h" header file):

```
O_RDONLY    Open for reading only.
O_WRONLY    Open for writing only.
```

O_RDWR Open for reading and writing.

For example, to open "data" for writing, assuming that the file had been created by another program, the following statements would be used:

```
int fd;  
fd = open( "data", O_WRONLY );
```

A few additional comments before proceeding:

- A "creat()" call implies an "open()". There is no need to "creat()" a file and then "open()" it.
- There is an operating-system-dependent limit on the number of files that a program can have open at any one time.
- The file descriptor is no more than an arbitrary number that a program uses to distinguish one open file for another. When a file is closed, re-opening it again will probably not give it the same file descriptor.

The "close()" system call is very simple. All it does is "close()" an open file when there is no further need to access it. The "close()" system call has the syntax:

```
close( <file descriptor> );
```

The "close()" call returns a value of 0 if it succeeds, and returns -1 if it encounters an error.

The "unlink()" system call deletes a file. It has the syntax:

```
unlink( <file_name_string> );
```

It returns 0 on success and -1 on failure. Note: Even after unlink, you will be able to read / write using fd.

The "write()" system call writes data to an open file. It has the syntax:

```
write( <file descriptor>, <buffer>, <buffer length> );
```

The file descriptor is returned by a "creat()" or "open()" system call. The "buffer" is a pointer to a variable or an array that contains the data; and the "buffer length" gives the number of bytes to be written into the file.

While different data types may have different byte lengths on different systems, the "sizeof()" statement can be used to provide the proper buffer length in bytes. A "write()" call could be specified as follows:

```
float array[10];  
...  
write( fd, array, sizeof( array ) );
```

The "write()" function returns the number of bytes it actually writes. It will return -1 on an error.

The "read()" system call reads data from a open file. Its syntax is exactly the same as that of the "write()" call:

```
read( <file descriptor>, <buffer>, <buffer length> );
```

The "read()" function returns the number of bytes it *actually* returns. At the end of file it returns 0, or returns -1 on error.

Retrieved from "https://en.wikibooks.org/w/index.php?title=A_Little_C_Primer/C_File-IO_Through_System_Calls&oldid=4051823"